/compact と /clear の違い

Claude Code のコンテキストが 100% に近づいたとき、どちらを打つべきか

— まず、これだけ —

いまやっている作業を、このまま続ける？

「続ける／続けない」だけで、打つコマンドが決まります

YES — 続ける

/compact

要約して、
そのまま作業を続行

▶ 焦点を指定すると精度が上がる
`/compact` 認証まわりの修正に絞って

NO — 別件に移る

/clear

空にして、
新しい会話を開始

▶ 先に名前をつけておくと後で探せる
`/rename` →（あとで）`/resume`

会議は続けたまま、これまでの話を**議事録1枚に要約**して机の上を空けるイメージ

会議室を出て、**まっさらな新しい部屋**で次の議題を始めるイメージ（前の部屋は残る）

出典：Anthropic 公式ドキュメント（code.claude.com/docs）／Command Reference・Manage costs effectively・Explore the context window・How Claude Code works・Model configuration
2026年8月時点の内容です。Claude Code は更新が速いため、最新の挙動は各自 /help および公式ドキュメントでご確認ください。 ｜ @ichi_ai_lab

こんなとき　同じ作業を続けたい → `/compact` を打つ

01 /compact

要約して、同じ会話を続ける

BEFORE — コンテキストが満杯（100%）



▼ /compact

AFTER — 要約だけが残り、空きができる



土台＝システムプロンプト・CLAUDE.md など　会話履歴　読んだファイル

1 **直前の作業内容を覚えたまま**、続きから作業できる

ただし細かいやり取りは要約に丸められるため、途中の細部は失われます。

2 要約の焦点を**自分で指定できる**

例：`/compact 認証まわりの修正に絞って` —— 自動任せより精度が上がります。

3 実行には**トークンがかかる**

会話全体を読んで要約するため、コンテキストが大きいほど重いリクエストになります。100% ギリギリまで放置せず、早めに打つのがコツ。

> 公式ドキュメントの説明
>
> ```
> Free up context by summarizing the conversation so far.
> Optionally pass focus instructions for the summary.
> ```
>
> （これまでの会話を要約してコンテキストを空ける。要約の焦点を指示として渡せる）

こんなとき　別の作業に移る → /clear を打つ

02 /clear

空にして、新しい会話を始める

BEFORE — コンテキストが満杯（100%）



▼ /clear

AFTER — 会話はゼロ（土台は自動で読み直される）



土台＝システムプロンプト・CLAUDE.md など　会話履歴　読んだファイル

1 直前の作業内容は**忘れる**。まっさらから始まる

別のタスクに移るときは、これがむしろ利点になります。

2 消えるわけではなく、**/resume で前の会話に戻れる**

打つ前に **/rename** で名前をつけておくと、あとから探しやすくなります。

3 実行に**トークンはかからない**

公式の表現は「**/clear costs nothing**」。長いセッションを引きずるより、切り替えたほうが結果的に安く、精度も戻ります。

> 公式ドキュメントの説明
>
> `Start a new conversation with empty context. (…) To free up context while continuing the same conversation, use /compact instead.`
>
> （空のコンテキストで新しい会話を開始する。同じ会話を続けたままコンテキストを空けたい場合は /compact を使うこと）　※別名：/reset, /new

こんなとき　どちらか決めきれない　→　この表の「どの行が今つらいか」で決める

03 ならべて比較

6つの観点で見ると、性格の違いがはっきりします

くらべる観点	/compact	/clear
コンテキスト	要約に圧縮して空ける	ゼロにリセット
直前の作業内容	**覚えている** 要約として引き継がれる	**忘れる** まっさらな状態から始まる
トークン消費	**かかる** 会話全体を読んで要約するため、大きいほど重い	**かからない** 公式：/clear costs nothing
前の会話	そのまま継続	消えない。**/resume** で戻れる
/usage の累計	継続して積み上がる	新セッション扱いでリセット
使いどころ	**同じタスクを続けたいとき**	**無関係な別タスクに移るとき**

こんなときは、こっち

/compact	ログイン機能の実装が途中。コンテキストが埋まってきた
/compact	長いデバッグの最中。これまでの経緯を忘れられると困る
/clear	ログイン機能が終わったので、次は別案件の記事作成に移る
/clear	回答が的外れになってきた。前の話を引きずっている感じがする

こんなとき　まだ 100% じゃない　→　埋まる前に、この3つで先手を打つ

04 100% になる前にやること

ギリギリで慌てるより、余裕のあるうちに手を打つほうが速くて安い

1 /context

まず、何が容量を食っているか見る

コンテキストの中身を色分けグリッドで表示してくれます。原因が MCP なのか、読み込んだファイルなのか、CLAUDE.md が長すぎるのかが一目でわかります。

2 /autocompact 500k

自動コンパクトが走るタイミングを早める

上限に近づくと自動でコンパクトは走りますが、その閾値を自分で前倒しできます。巨大なコンテキストを一度に要約させないための設定です。

3 サブエージェントに任せる

大きな調査・大量のファイル読み込みを外に出す

サブエージェントは別のコンテキストで動き、終わったら要約だけを返します。本体のコンテキストを圧迫しないので、長時間の作業ほど効きます。

覚えておくこと

コンテキストは「気づいたら満杯」になるもの。
/context を定期的にのぞく癖をつけるだけで、詰まる前に手を打てます。

こんなとき　/compact したら指示が効かなくなった　→　消えた指示を疑う

05 要約後に残るもの・消えるもの

同じ「指示」でも、書いた場所によって生き残るかどうかが変わります

残る　自動で読み直される

○ **システムプロンプト**
そもそも会話履歴の一部ではないため、影響を受けません

○ **プロジェクト直下の CLAUDE.md**
ディスクから再度読み込まれます

○ **オートメモリ（MEMORY.md）**
ディスクから再度読み込まれます

○ **フック**
コードとして動くものなので、コンテキストとは無関係

消える　読み直されない

× **paths: 付きのルール**
対象のファイルをもう一度読むまで、指示が効かなくなります

× **サブフォルダの CLAUDE.md**
そのフォルダのファイルをもう一度読むまで、同様に効きません

△ **呼び出したスキルの本文**
再度読み込まれますが、1スキル 5,000 トークン／合計 25,000 トークンの上限があり、超えた分は古いものから削除されます

つまり、やること

ずっと守らせたいルールは、会話の中ではなく
プロジェクト直下の CLAUDE.md に書いておく。

こんなとき　なんだか精度が落ちてきた・コストが high　→　この3つを疑う

06 やりがちなミス

この3つを避けるだけで、精度もコストもかなり変わります

1 別タスクに移るのに /compact を打つ

無関係な要約をずっと引きずることになり、回答の精度もトークンコストも悪化します。**作業が変わったら、迷わず /clear。**

2 「/clear したら消える」と誤解して使わない

会話は消えません。**/resume で戻れます。**怖がって長いセッションを続けるほうが、毎メッセージにコストがかかり続けて損です。

3 100% ギリギリまで放置する

巨大なコンテキストのコンパクトは、それ自体が重いリクエストになります。**余裕のあるうちに打つのが正解。**

もう一歩 —— 知っておくと差がつく

自動コンパクト	上限に近づくと自動で走ります。仕組みは **/compact** と同じ
/usage	トークンの使用量と、長コンテキストなど「効いている要因」を確認できます
新規セッション	会話がない状態で **/compact** を打つと **Not enough messages to compact.** と出ます

この資料のまとめ

続けるなら /compact　／　切り替えるなら /clear
迷ったら「この作業、続ける？」と自分に聞く。